

MUSTAFACODE

Lecture 21

Object Oriented Programming (CS304)
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

Lecture 21: Object Oriented Programming (CS304)



Object Oriented Programming (CS304)

Lecture No.21 — Unary Operators & Type Conversion (Detailed Notes)

◆ 1. Unary Operators Overview

Unary operators woh operators hotay hain jo **sirf ek operand (object)** par work karte hain.

C++ mein important unary operators:

- `++` (increment)
- `--` (decrement)

Ye operators **built-in types** aur **user-defined classes** dono ke liye overload kiye ja sakte hain.

◆ 2. ++ aur -- for Pre-defined Types

◆ Post-increment (x++)

👉 Post-increment mein:

- Pehle purani value use hoti hai
 - Phir value increment hoti hai
-

Example:

```
int x = 1, y = 2;

cout << y++ << endl;
cout << y;
```

Output:

```
2
3
```

Explanation:

- `y++` ne pehle `2` print kiya
- phir `y = 3` ho gaya

✗ Invalid Examples:

```
y++++; // Error
y++ = x; // Error
```

👉 Reason:

`y++` rvalue return karta hai (assign nahi hota)

◆ Pre-increment (++x)

👉 Pre-increment mein:

- Pehle value increment hoti hai
- Phir updated value return hoti hai

Example:

```
int y = 2;

cout << ++y << endl;
cout << y;
```

Output:

```
3
3
```

◆ Complex Example:

```
int x = 2, y = 2;

+++y;

cout << y;
```

Output:

```
4
```

◆ 3. Unary Operators in User Defined Classes (Complex)

◆ Pre-increment Overloading

Member Function:

```
class Complex {
    double real, img;

public:
    Complex &operator++();
};
```

Implementation:

```
Complex &Complex::operator++() {  
    real = real + 1;  
    return *this;  
}
```

👉 Explanation:

- `real` ki value 1 se increase hoti hai
- `*this` reference return hota hai

◆ Non-Member Function:

```
Complex &operator++(Complex &h) {  
    h.real += 1;  
    return h;  
}
```

📌 Real Example:

```
Complex h1, h2, h3;  
  
++h1;  
  
++h1 = h2 + ++h3;
```

👉 Important:

- Pre-increment **lvalue return karta hai**
- Is liye assignment possible hota hai

◆ 4. Compiler Pre vs Post kaise identify karta hai?

Type	Function Signature
Pre-increment	<code>operator++()</code>
Post-increment	<code>operator++(int)</code>

👉 `int` dummy parameter = post-increment ka indicator

◆ 5. Post-increment Overloading

◆ Member Function:

```
class Complex {  
public:  
    Complex operator++(int);  
};
```

Implementation:

```
Complex Complex::operator++(int) {  
    Complex t = *this; // old value save  
    real += 1;         // increment current object  
    return t;          // return old value  
}
```

◆ Non-Member Function:

```
Complex operator++(const Complex &h, int) {  
    Complex t = h;  
    h.real += 1;  
    return t;  
}
```

📌 Example:

```
Complex h1;  
  
h1++;
```

✗ Invalid:

```
h3++ = h2 + h3++; // Error
```

👉 Reason:

post-increment temporary value return karta hai (rvalue)

◆ 6. Pre vs Post Decrement (--)

👉 Same logic apply hoti hai:

- ++ ki jagah --
 - Implementation same hoti hai
-

◆ 7. Type Conversion (Basic Concept)

◆ Implicit Conversion

Compiler automatically convert karta hai:

```
int f = 0.021;    // float → int
double g = 34;    // int → double
```

◆ Explicit Conversion

```
int g = (int)0.021;
double h = double(35);
```

◆ 8. User Defined Type Conversion

C++ classes mein 2 types conversions hoti hain:

1. Other type → current type
2. Current type → other type

◆ 9. Constructor Based Conversion (Other → Current Type)

Example:

```
class String {
public:
    String(int a);
    char *GetStringPtr() const;
};
```


Implementation:

```
String::String(int a) {  
    char array[15];  
    itoa(a, array, 10);  
  
    size = strlen(array);  
    bufferPtr = new char[size + 1];  
    strcpy(bufferPtr, array);  
}
```

Example:

```
int main() {  
  
    String s = 345;  
    cout << s.GetStringPtr();  
}
```

Output:

345

⚠ Problem

```
String s = 'A';
```

👉 Problem:

- 'A' ASCII = 65 ban jata hai
- unintended conversion ho jati hai

◆ 10. Keyword: explicit

👉 `explicit` automatic conversion rokta hai

```
class String {  
public:  
    explicit String(int);  
};
```

✗ Invalid:

```
String s;  
s = 'A'; // Error
```

✓ Valid:

```
String s1;  
s1 = String(101);  
  
String s2;  
s2 = (String)204;
```

◆ 11. Operator Overloading Type Conversion

General Syntax:

```
TYPE1::operator TYPE2();
```

Example:

```
class String {  
public:  
    operator int();  
    operator char *();  
};
```

Implementation:

```
String::operator int() {  
    return atoi(bufferPtr);  
}  
  
String::operator char *() {  
    return bufferPtr;  
}
```

Example:

```
String s("2324");  
  
cout << (int)s << endl;  
cout << (char *)s;
```

Output:

```
2324  
2324
```

◆ 12. User Defined Type Conversion (Other Classes)

```
class String {  
public:  
    operator Complex();  
    operator HugeInt();  
    operator IntVector();  
};
```

◆ 13. Drawbacks of Type Conversion Operator

```
String s("Fakhir");  
cout << s;
```

👉 Problem:

- Compiler automatic conversion kar deta hai
- Unexpected output (junk) aa sakta hai

◆ 14. Best Practice (Recommended)

👉 Type conversion operators avoid karo

👉 Instead explicit functions use karo

Improved Version:

```
class String {  
public:  
    String(char *);  
    int AsInt();  
};
```

Implementation:

```
int String::AsInt() {  
    return atoi(bufferPtr);  
}
```

Example:

```
String s("434");  
  
cout << s.AsInt();
```

Output:

```
434
```

Final Summary

✓ Unary Operators:

- Pre-increment: pehle increment, phir return
- Post-increment: pehle return, phir increment

✓ Key Concepts:

- `int dummy parameter` → post-increment identification
- Pre-increment → lvalue return karta hai
- Post-increment → temporary object return karta hai

✓ Type Conversion:

- implicit vs explicit conversion
- constructor-based conversion

- operator-based conversion
-

✓ Best Practice:

- `explicit` use karo
 - implicit conversions avoid karo
 - `AsInt()` jese functions prefer karo
-

MUSTAFA CO